BÁO CÁO GIỮA KỲ MÔN ỨNG DỤNG AI CHO NGÔN NGỮ

DEVELOPMENT MINIMALIST OF THE LLAMA2 MODEL

Nhóm sinh viên thực hiện:

- Nguyễn Công Cường – 23020338
- Nguyễn Đức Huy – 23020376
- Nguyễn Trần Huy – 23020378
- Ngô Đình Linh – 23020394

1. Mục tiêu bài toán

Mục tiêu của bài tập là hoàn thiện các thành phần lõi còn thiếu của Llama2 tối giản trong `llama.py`, `rope.py`, `classifier.py`, `optimizer.py` và đánh giá mô hình trên 3 setting bắt buộc: generate, zero-shot prompting, và fine-tuning với trọng số tiền huấn luyện `stories42M.pt`.

2. Phần nhóm đã thực hiện

Nhóm hoàn thiện đầy đủ 7 hàm `#todo` trong 4 file chính:

- `llama.Attention.forward`
- `llama.RMSNorm.norm`
- `llama.Llama.forward`
- `llama.Llama.generate`
- `rope.apply_rotary_emb`
- `optimizer.AdamW.step`
- `classifier.LlamaEmbeddingClassifier.forward`

Chi tiết triển khai:

- Trong `llama.py`:
  - `RMSNorm.norm` dùng công thức chuẩn hóa `x / sqrt(mean(x^2) + eps)` rồi nhân với tham số học `weight`.
  - `Attention.forward` thực hiện luồng `(B,T,D) -> q,k,v`, tách head, áp dụng RoPE, lặp `k/v` theo `n_rep` để hỗ trợ GQA, tính `softmax(qk^T/sqrt(d))`, sau đó gộp head và chiếu ngược về không gian ẩn.
  - `LlamaLayer.forward` dùng kiến trúc pre-norm residual gồm 2 nhánh: attention và feed-forward (SwiGLU).
  - `Llama.forward` đi qua token embedding -> stack transformer layers -> RMSNorm cuối -> projection logits, đồng thời trả về `hidden_states` để phục vụ classifier.
  - `Llama.generate` sinh tự hồi quy từng token, tự cắt context theo `max_seq_len`, dùng argmax khi `temperature=0` và sampling khi `temperature>0`.
- Trong `rope.py`:
  - `apply_rotary_emb` tách từng cặp chiều thành biểu diễn phức (real/imag), tính góc quay theo vị trí token, nhân pha để quay `query/key`, rồi ghép lại đúng shape ban đầu.

- Trong `classifier.py`:
  - `LlamaEmbeddingClassifier.forward` lấy hidden state của token cuối không phải padding (`pad_id=0`), qua dropout và linear head, trả về `log_softmax` cho NLL loss.
- Trong `optimizer.py`:
  - `AdamW.step` cập nhật `exp_avg`, `exp_avg_sq`, bias correction dạng efficient, cập nhật tham số bằng Adam và thêm decoupled weight decay có nhân learning rate.

Kiểm thử xác thực trước thực nghiệm:

- `python rope_test.py` để kiểm tra RoPE.
- `python optimizer_test.py` để kiểm tra AdamW.
- `python sanity_check.py` để kiểm tra tích hợp forward pass của Llama.

3. Cách chạy code

Chi tiết cài đặt và toàn bộ lệnh chạy được trình bày trong file `README.md`, mục `Cài đặt và chạy`.

4. Siêu tham số

Mô hình từ `config.py`: `vocab=32000`, `hidden=512`, `layers=8`, `heads=8`, `kv_heads=8`, `max_len=1024`.

Setting	Siêu tham số dùng để báo cáo
Prompt (SST, CFIMDB)	`batch_size=10`, `seed=1337`
Finetune SST	`epochs=5`, `lr=2e-5`, `batch_size=80`, `dropout=0.3`, `seed=1337`
Finetune CFIMDB	`epochs=5`, `lr=2e-5`, `batch_size=10`, `dropout=0.3`, `seed=1337`
Generate	`max_new_tokens=75`, `temperature ∈ {0.0, 1.0}`

Thử thêm ngoài cấu hình chính: `epochs=1` cho CFIMDB (so sánh tại Mục 7).

5. Kết quả chính

Accuracy tính từ các file output đã sinh trong thư mục nộp bài:

Dataset	Tập	Prompting Accuracy	Finetuning Accuracy
SST	Dev	0.1562	0.4178
SST	Test	0.1480	0.4502
CFIMDB	Dev	0.4898	0.8980
CFIMDB	Test	0.3791	0.5020

Nhận xét:

- Zero-shot prompting cho kết quả thấp, đặc biệt trên SST-5 (5 lớp).
- Fine-tuning cải thiện đáng kể, rõ nhất ở CFIMDB dev.

- Chế độ generate cho câu hợp ngữ pháp ở `temperature=0.0`; đa dạng hơn nhưng nhiễu hơn ở `temperature=1.0`.

6. Kết quả advanced (tùy chọn)

Accuracy của các file `advanced`:

Dataset	Tập	Advanced Accuracy
SST	Dev	0.4187
SST	Test	0.4448
CFIMDB	Dev	0.9184
CFIMDB	Test	0.5020

Ghi chú:

- Nhóm dùng hướng `advanced` chạy nhanh: huấn luyện `TF-IDF + LinearSVM` trên tập train, sau đó confidence-based override dự đoán finetuning ở bước suy luận.
- Mức độ thay đổi so với finetuning: SST đổi `229/1101` mẫu dev và `421/2210` mẫu test; CFIMDB đổi `17/245` mẫu dev và `36/488` mẫu test.

7. Thử siêu tham số khác mặc định

Nhóm có thử thay đổi số epoch cho CFIMDB (giữ nguyên `lr=2e-5`, `batch_size=10`, `dropout=0.3`):

Cấu hình	Dev Accuracy
`epochs=1` (checkpoint `finetune-1-2e-05.pt`)	0.5061
`epochs=5` (best run, `cfimdb-dev-finetuning-output.txt`)	0.8980

Kết quả cho thấy tăng số epoch từ 1 lên 5 giúp cải thiện mạnh trên tập dev, vì vậy nhóm chọn `epochs=5` cho kết quả cuối cùng.

8. Kết luận

Nhóm đã hoàn thiện đầy đủ các thành phần lõi và tái lập đúng pipeline của đề. Kết quả cho thấy zero-shot chỉ ở mức cơ bản, còn fine-tuning cải thiện rõ rệt trên cả SST và CFIMDB (đặc biệt ở dev). Cấu hình chốt: `epochs=5`, `lr=2e-5`, batch size theo từng dataset.